

Set之三



SortedSet和NavigableSet

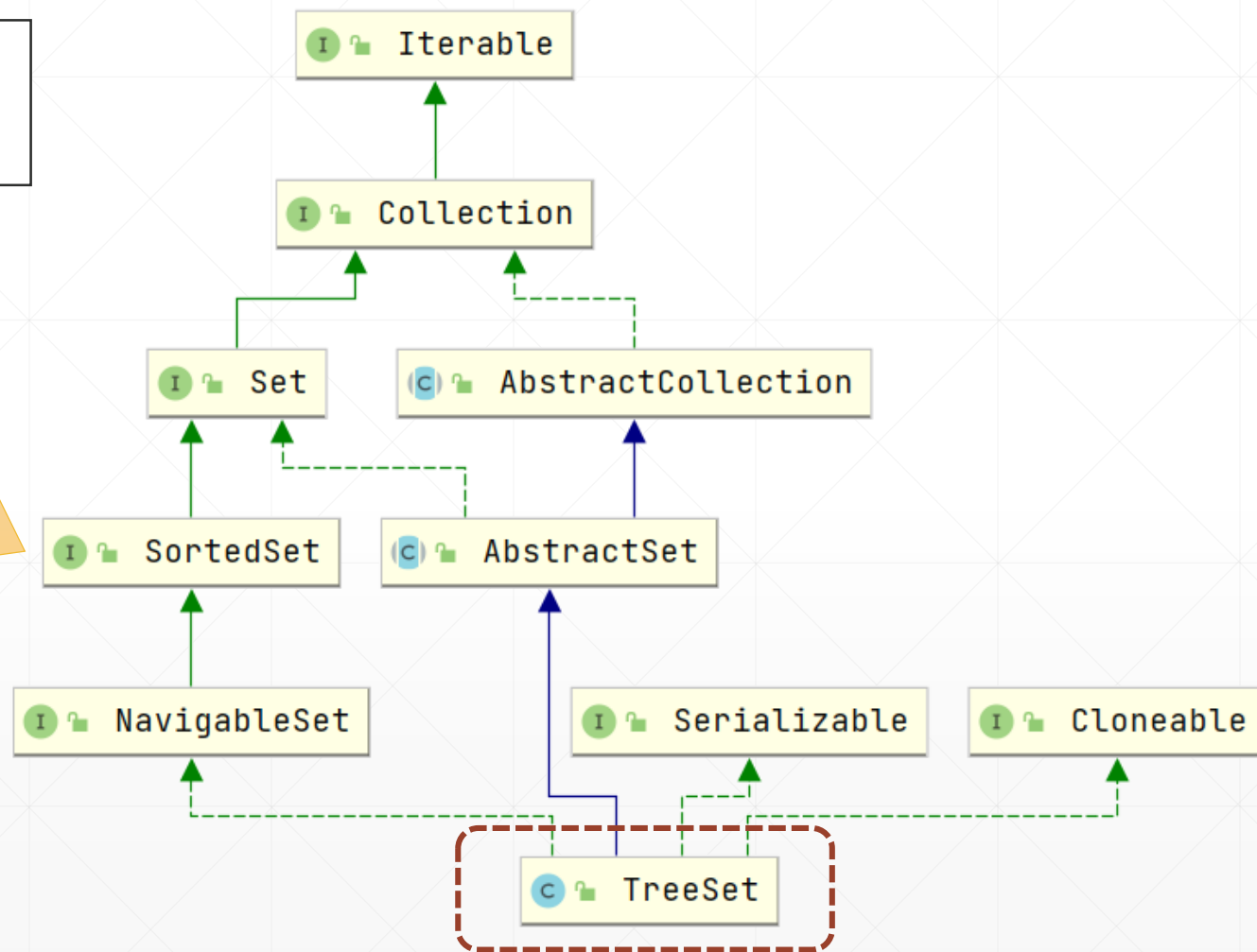
北京理工大学计算机学院
金旭亮

“恐怖”的TreeSet

TreeSet是一个功能强大的集合，但它的类型继承树也很恐怖.....

- SortedSet保证总是按照升序方式访问集合中的数据。
- 使用SortedSet的一个最大好处是：如果合并两个这样的集合，合并之后，集合仍然是有序的。

NavigableSet，能够查找指定元素“最相近”的元素。



SortedSet与NavigableSet接口

SortedSet接口，让元素有序：

```
E first();  
E last();  
  
SortedSet<E> tailSet(E fromElement);  
SortedSet<E> headSet(E toElement);  
SortedSet<E> subSet(E fromElement, E toElement);
```

NavigableSet接口，派生自SortedSet接口，添加了导航功能：

```
E lower(E e);  
E higher(E e);  
E floor(E e);  
E ceiling(E e);  
E pollFirst();  
E pollLast();
```

TreeSet同时实现了SortedSet和NavigableSet这两个接口，其用法与TreeMap高度类似，因为它内部使用的就是TreeMap进行数据存储的。

TreeSet的特性



TreeSet这个集合的最大特性是：

不管你向其中以什么样的顺序插入或删除元素，这些元素在TreeSet内部总是会被自动排序，因此，在任何时刻访问TreeSet时，你总能得到一个“已经排好序的对象集合”。



TreeSet内部调用`compareTo(Object obj)`方法来比较集合中各个对象的大小，然后在内部使用**红黑树**进行排序。

TreeSet的示例

UseTreeSetCommon.java
示例展示了TreeSet的基本使用方法。

```
UseTreeSetCommon x
"C:\Program Files\Java\jdk-15\bin\java.exe"
集合元素: [-9, 2, 5, 10]
集合里的第一个元素: -9
集合里的最后一个元素: 10
10之前的那个元素是: 5
10之后的那个元素是: null
floor(6)=5
ceiling(6)=10
返回小于4的子集, 不包含4: [-9, 2]
返回大于5的子集: [5, 10]
返回大于等于-3, 小于4的子集: [2]

Process finished with exit code 0
```

UseTreeSetCommon.java

```
//向TreeSet中添加四个Integer对象
var nums = new TreeSet<Integer>(List.of(5, 2, 10, -9));
//输出集合元素, 看到集合元素已经处于排序状态
System.out.println("集合元素:" + nums);
//输出集合里的第一个元素
System.out.println("集合里的第一个元素:" + nums.first());
//输出集合里的最后一个元素
System.out.println("集合里的最后一个元素:" + nums.last());
//可以输出指定元素的“前一个”和“后一个”
//练习: 如果lower()方法指定的元素在TreeSet中不存在, 会出现什么情况?
//编程试一试!
System.out.println("10之前的那个元素是: " + nums.lower(e: 10));
System.out.println("10之后的那个元素是: " + nums.higher(e: 10));
System.out.println("floor(6)=" + nums.floor(e: 6));
System.out.println("ceiling(6)=" + nums.ceiling(e: 6));
//返回小于4的子集, 不包含4
System.out.println("返回小于4的子集, 不包含4:" + nums.headSet(toElement: 4));
//返回大于5的子集, 如果Set中包含5, 子集中还包含5
System.out.println("返回大于5的子集:" + nums.tailSet(fromElement: 5));
//返回大于等于-3, 小于4的子集。
System.out.println("返回大于等于-3, 小于4的子集:" + nums.subSet(-3, 4));
```

TreeSet对元素的要求-1

右侧示例代码，canNotAdd()方法在运行时将导致抛出ClassCastException.....

TestTreesetError x

```
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Program Files\
Exception in thread "main" java.lang.ClassCastException Create breakpoint : cl
    at java.base/java.util.TreeMap.compare(TreeMap.java:1563)
    at java.base/java.util.TreeMap.addEntryToEmptyMap(TreeMap.java:768)
    at java.base/java.util.TreeMap.put(TreeMap.java:777)
    at java.base/java.util.TreeMap.put(TreeMap.java:534)
    at java.base/java.util.TreeSet.add(TreeSet.java:255)
    at TestTreesetError.showError(TestTreesetError.java:16)
    at TestTreesetError.main(TestTreesetError.java:6)
```

```
static class Err {
    int value;
}

static void canNotAdd() {
    var ts = new TreeSet<Err>();
    //尝试向TreeSet集合中添加一个Err对象
    ts.add(new Err());
}
```

TestTreesetError.java

TreeSet对元素的要求-2



TreeSet要求其元素必须实现Comparable接口，并且应该是同一个类型的，否则，将引发ClassCastException。

前述示例中，Err类没有实现这个接口。



```
static class Err {  
    int value;  
}
```



修正

```
static void canAdd(){  
    var ts = new TreeSet<Err2>();  
    //尝试向TreeSet集合中添加一个Err2对象  
    ts.add(new Err2());  
}
```



//让自身可以比较大小

```
static class Err2 implements Comparable<Err2>{  
    int value;  
    @Override  
    public int compareTo(Err2 o) {  
        return Integer.compare(value, o.value);  
    }  
}
```

现在上述代码就没有问题了！

仅能“比较大小”，那还是不够的！

```
static void canNotAdd2() {  
    var ts = new TreeSet();  
    //向TreeSet集合中添加两个不同类型的对象  
    ts.add("example string");  
    ts.add(new Date());  
}
```



```
Run: TestTreeSetError x  
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Pr  
Exception in thread "main" java.lang.ClassCastException Create  
    at java.base/java.util.Date.compareTo(Date.java:133)  
    at java.base/java.util.TreeMap.put(TreeMap.java:806)  
    at java.base/java.util.TreeMap.put(TreeMap.java:534)  
    at java.base/java.util.TreeSet.add(TreeSet.java:255)  
    at TestTreeSetError.canNotAdd2(TestTreeSetError.java:41)  
    at TestTreeSetError.main(TestTreeSetError.java:8)
```



想加入到TreeSet中的对象，还必须是同一类型的！

如果对象没有实现Comparable接口.....



对于那些没有实现Comparable接口的对象，如果想把它放到TreeSet中，可以自定义一个Comparator，并把它传给TreeSet即可。

TreeSet有一个构造方法，可以接收一个Comparator对象，其定义如下：

```
public TreeSet(Comparator<? super E> comparator) {  
  
}
```

使用TreeSet保存自身不支持大小比较的对象

CommonClass 类有两个属性：info和num，但它自身不直接支持大小比较。为此，为它定制了两个比较器，一个使用info属性值进行比较，另一个使用num属性值进行比较。

CommonClass	
f	BY_NUMS Comparator<CommonClass>
f	BY_INFO Comparator<CommonClass>
m	toString() String
m	equals(Object) boolean
m	hashCode() int
p	info String
p	num int

//一个用于比较CommonClass对象的第三方比较器，使用的是最原始的匿名类型语法
//可以使用Lambda表达式进行简化

```
public static final Comparator<CommonClass> BY_NUMS =  
    comparing(new Function<CommonClass, Integer>() {  
        @Override  
        public Integer apply(CommonClass myClass) {  
            return myClass.num;  
        }  
    });
```

//使用Lambda表达式特性简化后的表达

```
public static final Comparator<CommonClass> BY_INFO=  
    comparing(CommonClass::getInfo);
```

使用示例

在创建TreeSet时，通知它使用按照nums属性值进行排序的比较器对象，从输出结果来看，确实如此。

```
public class CommonClassAndTreeSet {  
    public static void main(String[] args) {  
  
        //指定TreeSet使用一个Comparator对象  
        var treeSet = new TreeSet<CommonClass>(CommonClass.BY_NUMS);  
        Random ran = new Random();  
        //向集合中加入3个元素  
        for (int i = 0; i < 3; i++) {  
            var obj = new CommonClass();  
            int ranValue = ran.nextInt( bound: 100);  
            obj.setNum(ranValue);  
            obj.setInfo("info " + ranValue);  
            treeSet.add(obj);  
        }  
        //输出:  
        // [MyClass{nums=35, info='info 35'},  
        //  MyClass{nums=46, info='info 46'},  
        //  MyClass{nums=75, info='info 75'}]  
        System.out.println(treeSet);  
    }  
}
```

使用TreeSet要注意的地方.....



1

放到TreeSet中的对象，应该是可比较大小的，并且应该属于同一种类型。

2

当把一个对象放入TreeSet中，并且你重写了该对象的equals()方法，那么，你应该保证该方法与compareTo(Object obj)方法有一致的结果。

3

已经放到Set中的对象，不要修改它的属性值！

课后练习：注意避开TreeSet的陷阱



- 当把一个对象加入TreeSet集合时，TreeSet调用对象的compareTo()方法与集合中的其他对象比较大小，然后根据红黑树算法决定它的存储位置。
- 要注意：
 - 对象的compareTo方法与equals方法的返回值必须一致，否则会带来混乱。
 - 如果修改TreeSet集合中对象的属性，导致原先不等的对象“相等”了，同样可能会造成混乱。
- 请阅读并运行WhatIsWrongWithTreeSet.java和WhatIsWrongWithTreeSet示例，理解上述特性。

下一讲



集合方法 (Collection Methods)